INTRODUCTION TO DEEP LEARNING AND GENAI

PROJECT REPORT

Smart MCQ Solver

Deep Learning Based Multiple Choice Answer Ranking

Name : Nihal Ahamed

Roll No : 23f2000003

Final Kaggle Leaderboard Score : 0.75062

Deep Learning and GenAI Project

1. Abstract

This project was carried out for the Smart MCQ Solver Challenge, where the task is to predict the three most likely answers for a multiple-choice question in ranked order. Each question contains a prompt and five answer choices labelled A, B, C, D and E. The main evaluation metric used by the competition is Mean Average Precision at 3 (MAP@3), so the position of the correct answer matters.

I started with a simple TF-IDF cosine similarity method to get a baseline. After that, I implemented a BiLSTM with Attention from scratch and then compared it with two pretrained transformer models, DistilBERT and RoBERTa. The models were trained on the same training-validation split and their performance was tracked using Weights & Biases.

The baseline MAP@3 was 0.2904. The BiLSTM reached 0.9950 MAP@3 on the validation set, while DistilBERT and RoBERTa reached 0.9504 and 0.9658 respectively. The BiLSTM was therefore selected as the best model on the validation set. The final submission was generated from the selected model and obtained a Kaggle leaderboard score of 0.75062. The difference between the validation result and the leaderboard score is an important observation from this project, as it shows that a very strong validation result does not always translate directly to unseen competition data.

2. Introduction

2.1 Problem Statement

Multiple choice question answering looks simple from the outside, but a model has to understand the question, compare all five choices and decide which choice is most appropriate. In this competition, the model is not asked to return only one answer. It must return the top three choices in order of confidence.

For example, if the correct answer is A, the prediction A B C is better than B A C because the correct answer appears at the first position. This makes the task a combination of classification and ranking.

Aspect	Description
Task	Multiple Choice Question Answer Ranking
Input	Question prompt and five answer choices
Output	Top three answer labels in ranked order
Answer labels	A, B, C, D, E
Training samples	2,000
Test samples	500
Primary metric	MAP@3
Other metrics	Accuracy and Macro F1

2.2 Project Objectives

- Understand the structure of the MCQ dataset and prepare it for deep learning.
- Build a simple TF-IDF based baseline before moving to neural networks.
- Implement a BiLSTM with Attention from scratch.
- Fine-tune DistilBERT and RoBERTa for the multiple-choice task.
- Compare the models using Accuracy, Macro F1 and MAP@3.
- Track the main experiments using Weights & Biases.
- Select the model with the highest validation MAP@3 for the final prediction.
- Generate the required top-3 Kaggle submission file.

2.3 Report Structure

Section 3 describes the dataset and preprocessing. Section 4 explains the models and the reason for using each one. Section 5 discusses the experiments and results. Section 6 explains the final prediction and Kaggle submission. The last section summarizes what was learned and possible improvements.

3. Dataset and Preprocessing

3.1 Dataset Description

The dataset contains academic multiple-choice questions. The questions cover different topics, including physics, astronomy, philosophy and other technical subjects. The training file contains the correct answer for each question, while the test file does not contain the answer column.

Column	Description
id	Unique question identifier
prompt	Question or problem statement
A	First answer choice
B	Second answer choice
C	Third answer choice
D	Fourth answer choice
E	Fifth answer choice
answer	Correct answer label in training data

3.2 Dataset Exploration

The training data contains 2,000 rows and eight columns. All eight columns are complete, so there were no missing values that required imputation. The test set contains 500 questions.

Answer	Count
A	369
B	490
C	459
D	358
E	324

The answer distribution is not perfectly balanced. B is the most frequent answer and E is the least frequent one, but every class has a reasonable number of examples. I therefore kept the original labels and used a stratified split so that the validation set would retain a similar distribution.

The initial text analysis found 37,072 total words and 875 unique vocabulary items in the directly examined prompt corpus. For the custom neural network, a separate training vocabulary was built, resulting in a vocabulary size of 3,798 after applying the frequency rule.

3.3 Text Construction

For the neural and transformer models, the question and its five options were combined into a single text representation. The fields were separated using a [SEP] marker. This gives the model the complete context needed to compare the question with all the answer choices.

$$prompt + " [SEP] " + A + " [SEP] " + B + " [SEP] " + C + " [SEP] " + D + " [SEP] " + E$$

3.4 Train-Validation Split

The training data was split into 80% training and 20% validation data using a fixed random state of 42 and stratification on the answer label.

Split	Number of samples
Training	1600
Validation	400
Test	500

4. Modeling and Experimentation

I used the models in stages rather than starting directly with a large transformer. The baseline gave a simple reference point, the BiLSTM showed what could be achieved with a model built from scratch, and the two transformer models gave a comparison with pretrained language representations.

4.1 TF-IDF Cosine Similarity Baseline

The first approach was intentionally simple. TF-IDF was used to represent the question and each answer choice as vectors. Cosine similarity was then calculated between the prompt and each option. The five options were sorted by similarity and the top three were returned.

The reason for starting with TF-IDF was not to build the final system, but to understand how far a simple word-overlap method could go. It also provides a useful reference when evaluating the deep learning models.

Baseline MAP@3: 0.2904

4.2 BiLSTM with Attention

The first deep learning model was a Bidirectional LSTM with an Attention layer. It was built from scratch, including the tokenizer and vocabulary. This was important because the project requirement included a model built without starting from a pretrained language model.

- Vocabulary size: 3,798
- Embedding dimension: 128
- Hidden dimension: 128
- Maximum sequence length: 256
- Dropout: 0.5
- Batch size: 32
- Epochs: 5
- Optimizer: Adam
- Learning rate: 0.001
- Loss: CrossEntropyLoss
- Weight decay: 0.00001

A BiLSTM reads the sequence in both directions, which gives it information from the words before and after a particular position. Attention was added so that the network could give more importance to the parts of the text that were useful for the final classification.

4.3 BiLSTM Training Results

Epoch	Train Loss	Validation Loss	Accuracy	F1
1	1.5672	1.4880	0.3250	0.1862
2	1.0801	0.6956	0.7600	0.7594
3	0.4625	0.2406	0.9075	0.9022
4	0.2243	0.0824	0.9800	0.9792
5	0.0679	0.0318	0.9900	0.9891

The training curve shows a large improvement during the first few epochs. By epoch 5, validation accuracy was 0.9900 and the validation F1 score was 0.9891. The final validation MAP@3 was 0.9950.

4.4 DistilBERT

After the custom BiLSTM, I used DistilBERT as a pretrained model. The advantage here is that the transformer already contains language representations learned during large-scale pretraining. Instead of learning all language patterns from the 2,000 questions, the model can fine-tune those existing representations for the MCQ task.

- Base model: distilbert-base-uncased
- Task head: DistilBertForMultipleChoice
- Maximum sequence length: 256
- Training epochs: 3
- Batch size: 4
- Learning rate: 2e-5
- Weight decay: 0.01

The five answer choices are considered together with the question. The multiple-choice head produces a score for each of the five options, which can then be ranked to produce the required top-three prediction.

4.5 DistilBERT Results

Metric	Value
Accuracy	0.9225
Macro F1	0.9218
MAP@3	0.9504
Evaluation loss	0.68071
Evaluation runtime	8.6862 seconds
Evaluation samples/second	46.05
Evaluation steps/second	5.756

The DistilBERT run was tracked in WandB under the name DistilBERT_Run. The WandB logs also recorded the training epoch, global step, learning rate, gradient-related information and evaluation performance.

4.6 RoBERTa

RoBERTa was used as the third model. It is another pretrained transformer, but it is larger than DistilBERT. The purpose was to see whether a stronger pretrained representation could improve the ranking performance on the same validation set.

- Base model: roberta-base
- Task head: RobertaForMultipleChoice
- Maximum sequence length: 256

- Training epochs: 3
- Batch size: 4
- Learning rate: 2e-5
- Weight decay: 0.01

The training and evaluation process was kept similar to DistilBERT so that the comparison would mainly reflect the difference between the two pretrained architectures.

4.7 RoBERTa Results

Metric	Value
Accuracy	0.9475
Macro F1	0.9462
MAP@3	0.9658
Evaluation loss	0.68071
Evaluation runtime	8.6862 seconds
Evaluation samples/second	46.05
Evaluation steps/second	5.756

RoBERTa performed better than DistilBERT on the validation set. The final WandB run also recorded the test-side runtime and throughput measurements shown in the experiment dashboard.

4.8 Weights & Biases Tracking

Weights & Biases was used to keep the training runs together instead of relying only on notebook output. The project contains separate runs for BiLSTM_Run, DistilBERT_Run and RoBERTa_Run. For the transformer runs, the dashboard records training and evaluation loss, epoch, global step, learning rate, runtime and throughput.

The screenshots included in this report are the actual WandB outputs from the final notebook run. They are included to show the training behaviour and the recorded experiment values rather than adding artificial charts.

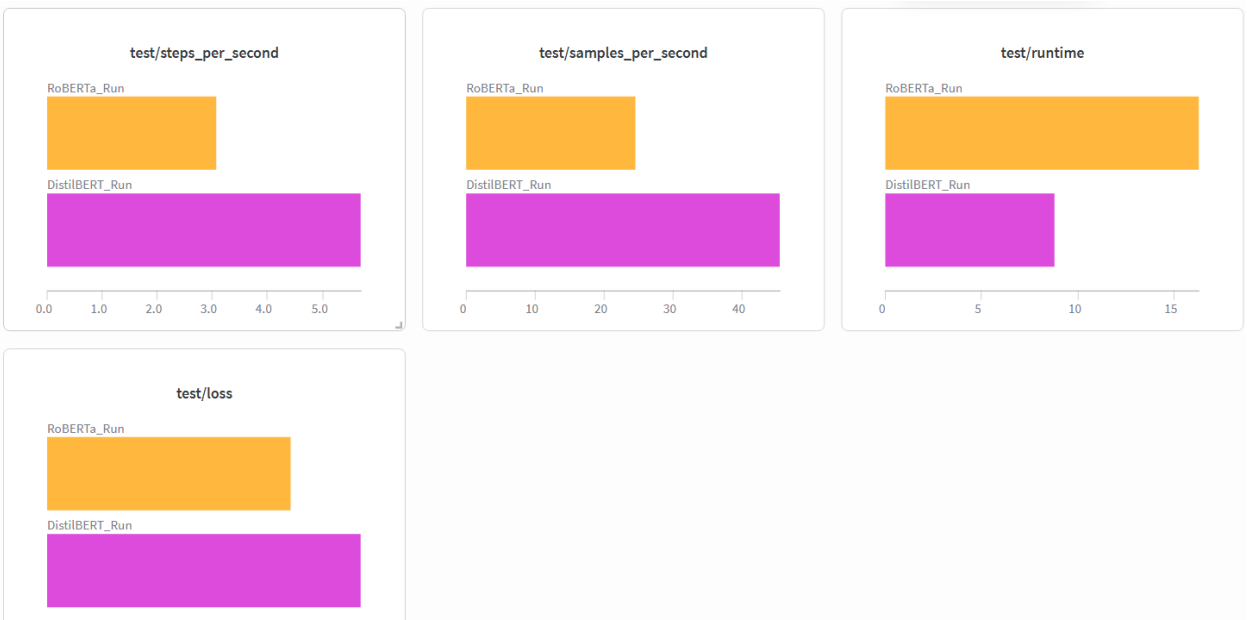


Figure 1. WandB training and test-side metrics for the transformer runs.

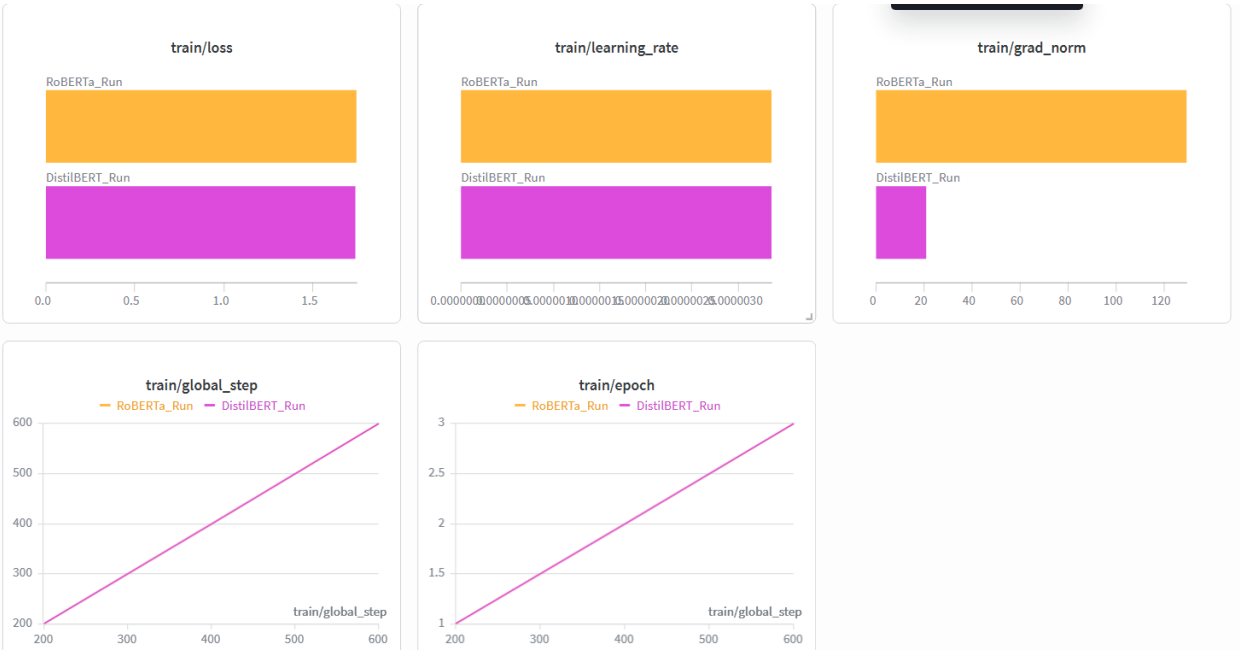


Figure 2. WandB training loss, learning rate, gradient norm and epoch tracking.

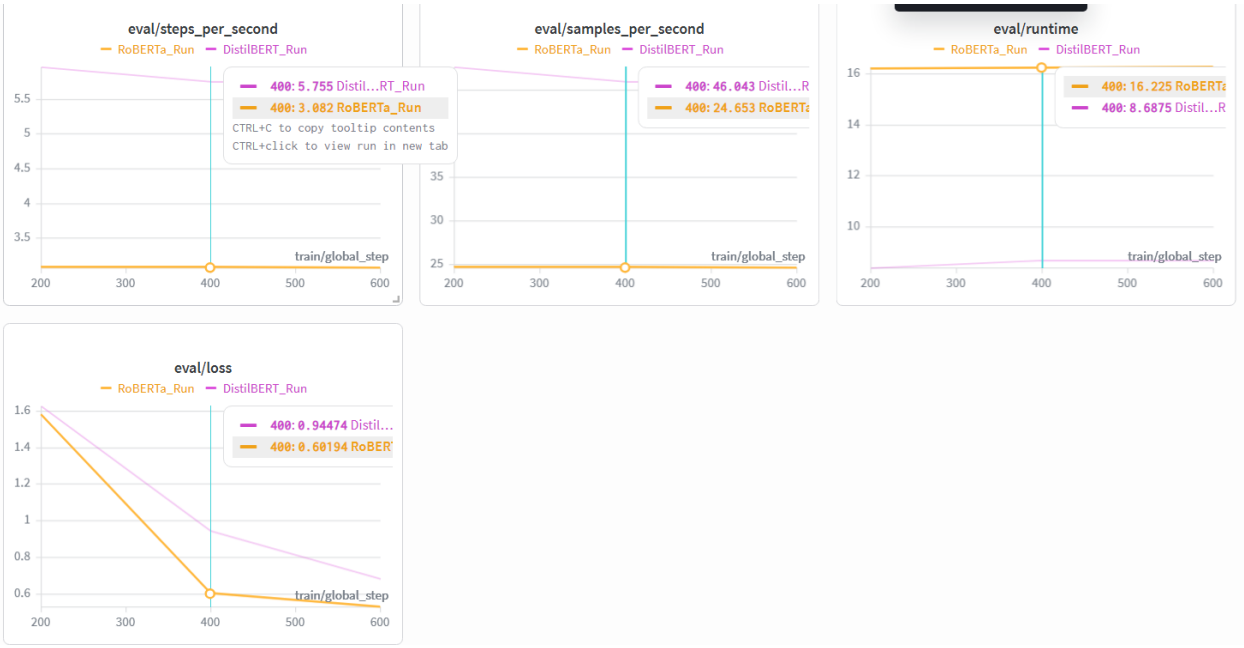


Figure 3. WandB evaluation throughput and evaluation loss comparison.

5. Results and Performance Analysis

5.1 Evaluation Metrics

Accuracy was used to measure whether the highest-ranked answer was correct. Macro F1 was used to account for the five answer classes without allowing the most frequent class to dominate the result. MAP@3 was the main competition metric because the final submission contains three ranked answers.

5.2 Model-wise Comparison

Model	Type	Accuracy	Macro F1	Validation MAP@3
TF-IDF	Baseline	-	-	0.2904
BiLSTM + Attention	From scratch	0.9900	0.9891	0.9950
DistilBERT	Pretrained	0.9225	0.9218	0.9504
RoBERTa	Pretrained	0.9475	0.9462	0.9658

The BiLSTM obtained the highest validation MAP@3 at 0.9950, followed by RoBERTa at 0.9658 and DistilBERT at 0.9504. The TF-IDF baseline was much lower at 0.2904. Based on this validation comparison, the notebook selected BiLSTM as the best model.

5.3 What the Results Show

The baseline result was useful because it showed that matching similar words between a question and an answer is not enough for this dataset. The jump from 0.2904 to the neural models was large.

The BiLSTM result was especially strong on the validation set. Its validation accuracy increased from 0.3250 in the first epoch to 0.9900 by the fifth epoch. The corresponding F1 score increased from 0.1862 to 0.9891. This shows that the model learned the training patterns quickly.

Among the transformer models, RoBERTa performed better than DistilBERT. Its accuracy was 0.9475 compared with 0.9225 for DistilBERT, and its MAP@3 was 0.9658 compared with 0.9504.

5.4 Validation Score vs Kaggle Score

One of the most important observations from the final project is the gap between the validation results and the actual Kaggle leaderboard result. The selected BiLSTM had a validation MAP@3 of 0.9950, but the final Kaggle leaderboard score was 0.75062.

Result	MAP@3
TF-IDF validation	0.2904
BiLSTM validation	0.9950
DistilBERT validation	0.9504
RoBERTa validation	0.9658
Final Kaggle leaderboard	0.75062

This suggests that the validation split was easier or did not fully represent the distribution of the hidden test questions. It is also a reminder that model selection based on one validation split can sometimes give an overly optimistic estimate. In a competition setting, this is an important limitation to discuss rather than hiding the difference.

6. Final Prediction and Submission

6.1 Model Selection

The notebook compares the three validation MAP@3 scores and selects the model with the highest value. The scores were 0.9950 for BiLSTM, 0.9504 for DistilBERT and 0.9658 for RoBERTa. Therefore, BiLSTM was selected.

6.2 Top-3 Prediction Generation

For each test question, the selected model produces scores for the five answer labels. The scores are sorted from highest to lowest, and the first three labels are written in the required order. For example, if the model scores A, C and B highest, the submission entry becomes A C B.

6.3 Submission File

The final notebook successfully created submission.csv. The file contains the question ID and a prediction field containing three space-separated answer labels. The final Kaggle leaderboard score for this submission was 0.75062.

7. Conclusion and Future Work

In this project, I built a complete pipeline for the Smart MCQ Solver problem, starting with a simple text similarity baseline and moving through a custom recurrent neural network and two pretrained transformer models.

The TF-IDF baseline gave a MAP@3 of 0.2904. The BiLSTM with Attention gave the best validation result at 0.9950 MAP@3, with 0.9900 validation accuracy and 0.9891 F1 after five epochs. DistilBERT reached 0.9504 MAP@3 and RoBERTa reached 0.9658. Based on these validation results, BiLSTM was selected for the final submission.

The final Kaggle score was 0.75062. Although this was lower than the validation score, the result was useful because it exposed the difference between a fixed validation split and the hidden competition distribution. This was one of the main practical lessons from the project.

7.1 Future Improvements

- Use K-fold cross-validation instead of relying on a single 80:20 validation split.
- Investigate why the validation performance of the BiLSTM was much higher than the Kaggle leaderboard score.
- Use stronger multiple-choice architectures that explicitly score each question-option pair.
- Experiment with DeBERTa and other pretrained language models.
- Try parameter-efficient fine-tuning methods such as LoRA for transformer models.
- Combine model probabilities more carefully instead of selecting only one model.
- Perform error analysis on questions where the models disagree.
- Tune sequence length, learning rate, batch size and regularization using a validation strategy that is less sensitive to one split.